

OpenCV-Based Real-Time Vision Systems

*Pseudocode Design for (1) Face Detection & Recognition and (2) Vehicle Detection, Tracking
& Counting*

1. Real-Time Face Detection and Recognition

This section presents a complete OpenCV-based pipeline for a security system that captures a live camera stream, detects multiple faces simultaneously, recognizes registered persons from a trained dataset, and displays bounding boxes with identity labels — all while operating in real time with minimized processing delay.

1.1 Pipeline Overview

Real-Time Face Detection & Recognition Pipeline

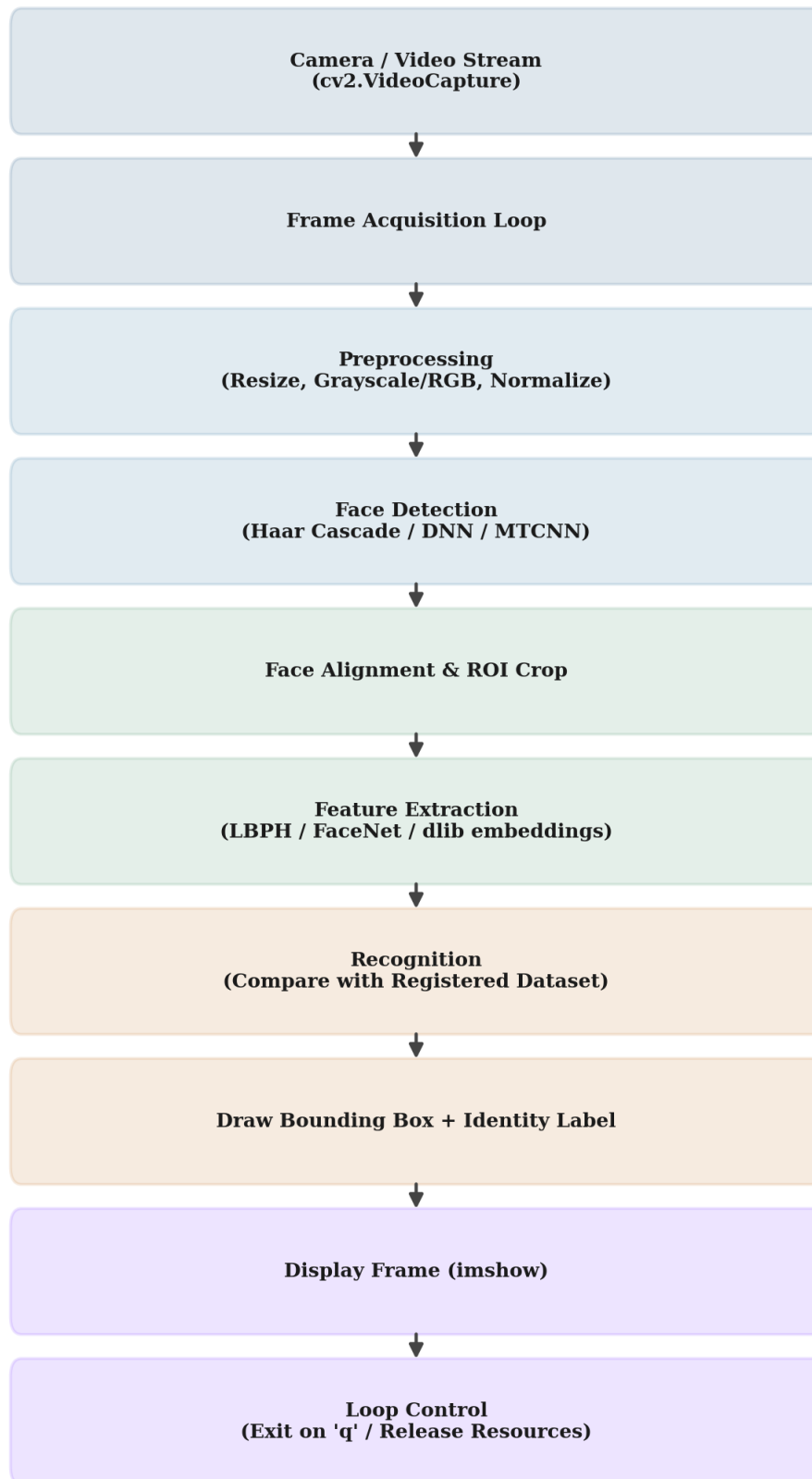


Figure 1: End-to-end real-time face detection and recognition pipeline.

1.2 Detailed Pseudocode

```
BEGIN FaceDetectionRecognitionSystem
```

```
    LOAD face_detector_model
```

```
    LOAD face_recognizer_model
```

```
    LOAD registered_dataset
```

```
    SET frame_skip_counter = 0
```

```
    SET FRAME_SKIP = 2
```

```
    SET RECOGNITION_THRESHOLD = 0.55
```

```
    OPEN video_stream = VideoCapture(source = camera_index OR RTSP_URL)
```

```
    IF video_stream NOT opened THEN
```

```
        PRINT "Error: cannot access camera"
```

```
        EXIT PROGRAM
```

```
    END IF
```

```
    INITIALIZE fps_counter, prev_time
```

```
    WHILE video_stream IS opened DO
```

```
        SUCCESS, frame = video_stream.read()
```

```
        IF NOT SUCCESS THEN
```

```
            BREAK
```

```
        END IF
```

```
        frame_skip_counter = frame_skip_counter + 1
```

```
        // ---- 2a. PREPROCESSING ----
```

```
        resized_frame = RESIZE(frame, width = 640)      // shrink for speed
```

```
        gray_frame  = CONVERT_TO_GRAYSCALE(resized_frame) // for Haar-based  
detectors
```

```
        rgb_frame   = CONVERT_BGR_TO_RGB(resized_frame) // for DNN-based detectors
```

```
        NORMALIZE(rgb_frame)                          // scale pixel values
```

```

// ---- 2b. SKIP FRAMES FOR SPEED (optional throttling) ----
IF frame_skip_counter MOD FRAME_SKIP != 0 THEN
    DISPLAY(frame)                // show last known boxes
    IF KEY_PRESSED('q') THEN BREAK
    CONTINUE
END IF

// ---- 2c. FACE DETECTION (multiple faces) ----
detections = face_detector_model.detect(gray_frame OR rgb_frame)
// detections = list of (x, y, w, h, confidence)

FILTER detections WHERE confidence >= MIN_DETECTION_CONFIDENCE

face_boxes = []
FOR EACH (x, y, w, h, confidence) IN detections DO
    APPEND (x, y, w, h) TO face_boxes
END FOR

// ---- 2d. PARALLEL / BATCH PROCESSING OF FACES ----
results = []                    // (box, identity, score)

FOR EACH box IN face_boxes DO    // handles multiple faces
    (x, y, w, h) = box

    // ---- Face alignment & ROI extraction ----
    face_roi = CROP(rgb_frame, x, y, w, h)
    face_roi = ALIGN_FACE(face_roi)        // eye-landmark based alignment
    face_roi = RESIZE(face_roi, 160, 160)  // standard input size

    // ---- Feature extraction ----
    embedding = face_recognizer_model.extract_features(face_roi)
    // e.g., 128-D / 512-D embedding vector

    // ---- Recognition: compare against registered dataset ----

```

```

best_match_name = "Unknown"
best_match_score = INFINITY

FOR EACH (known_embedding, known_name) IN registered_dataset DO
    distance = COMPUTE_DISTANCE(embedding, known_embedding) // cosine /
euclidean
    IF distance < best_match_score THEN
        best_match_score = distance
        best_match_name = known_name
    END IF
END FOR

IF best_match_score > RECOGNITION_THRESHOLD THEN
    best_match_name = "Unknown"
END IF

APPEND (box, best_match_name, best_match_score) TO results
END FOR

// ---- 2e. VISUALIZATION ----
FOR EACH (box, name, score) IN results DO
    (x, y, w, h) = box
    DRAW_RECTANGLE(frame, (x, y), (x + w, y + h), color = GREEN, thickness = 2)
    label = name + " (" + FORMAT(score, 2) + ")"
    DRAW_LABEL_BACKGROUND(frame, label, position = (x, y - 20))
    PUT_TEXT(frame, label, position = (x, y - 8), font_scale = 0.5, color = WHITE)
END FOR

// ---- 2f. PERFORMANCE OVERLAY ----
current_time = GET_TIME()
fps = 1 / (current_time - prev_time)
prev_time = current_time
PUT_TEXT(frame, "FPS: " + FORMAT(fps, 1), position = (10, 20))

// ---- 2g. DISPLAY ----

```

```

    DISPLAY(frame, window_name = "Face Recognition")

    // ---- 2h. EFFICIENT LOOP CONTROL / EXIT ----
    IF KEY_PRESSED('q') THEN
        BREAK
    END IF

END WHILE

// ----- 3. CLEANUP -----
video_stream.RELEASE()
DESTROY_ALL_WINDOWS()

END FaceDetectionRecognitionSystem

```

1.3 Key Design Notes for Real-Time Performance

- Downscaling frames (e.g., to 640px width) before detection substantially cuts per-frame cost.
- Frame skipping / alternating detection and tracking (e.g., detect every 3rd frame, use a lightweight tracker like KCF in between) reduces load while remaining responsive.
- A DNN-based detector (SSD/MTCNN) on GPU, or a Haar Cascade on CPU for lighter loads, should be chosen based on available hardware.
- Batching all detected faces through the feature extractor together (vectorized) rather than one-by-one improves throughput when multiple faces are present.
- The registered dataset should be pre-encoded into embeddings once (offline) so the loop only performs fast distance comparisons, not re-training.
- Releasing the capture device and destroying display windows on exit prevents resource leaks.

2. Vehicle Detection, Tracking and Counting

This section presents a complete OpenCV-based traffic monitoring pipeline that detects vehicles within a predefined region of interest, tracks multiple vehicles across consecutive frames, counts vehicles as they cross a virtual line, and displays bounding boxes, tracking IDs, and a running count — while avoiding double counting and maintaining near real-time performance.

2.1 Pipeline Overview

Vehicle Detection, Tracking & Counting Pipeline

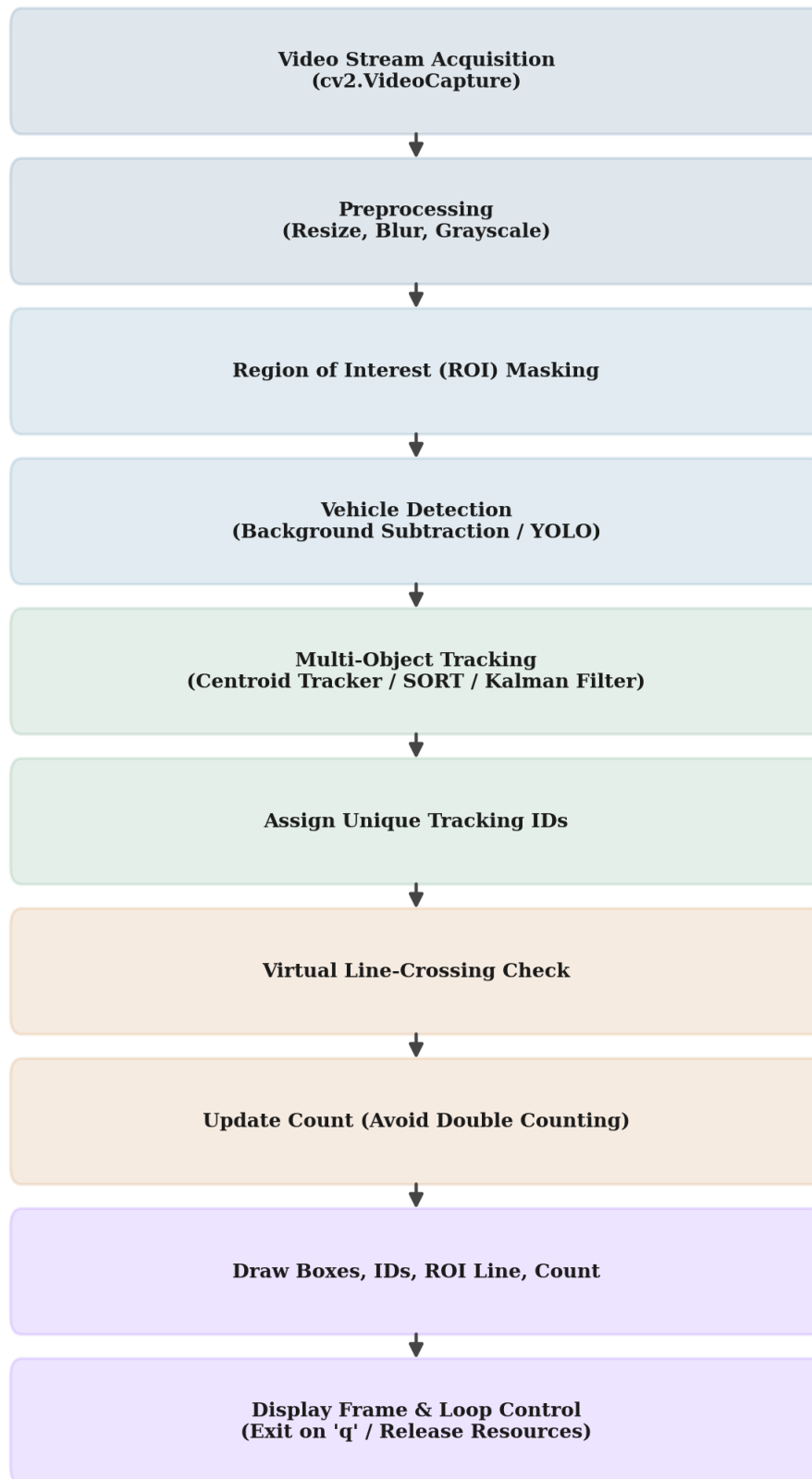


Figure 2: End-to-end vehicle detection, tracking, and counting pipeline.

2.2 Detailed Pseudocode

```
BEGIN VehicleDetectionTrackingCountingSystem
```

```
// ----- 1. INITIALIZATION -----
```

```
LOAD vehicle_detector_model      // YOLO / SSD / Background Subtraction (MOG2)
```

```
INITIALIZE tracker_registry = {} // maps tracking_id -> tracker state (centroid, box, age)
```

```
SET next_id = 0
```

```
SET vehicle_count = 0
```

```
SET counted_ids = SET()          // stores IDs already counted (avoid double counting)
```

```
DEFINE ROI_POLYGON               // region of interest coordinates
```

```
DEFINE COUNT_LINE = ((x1, y1), (x2, y2)) // virtual counting line
```

```
SET MAX_DISAPPEARED_FRAMES = 15 // frames a track can be missing before  
removal
```

```
SET IOU_MATCH_THRESHOLD = 0.3
```

```
OPEN video_stream = VideoCapture(source = video_file OR camera_index)
```

```
IF video_stream NOT opened THEN
```

```
    PRINT "Error: cannot open video stream"
```

```
    EXIT PROGRAM
```

```
END IF
```

```
// ----- 2. MAIN PROCESSING LOOP -----
```

```
WHILE video_stream IS opened DO
```

```
    SUCCESS, frame = video_stream.read()
```

```
    IF NOT SUCCESS THEN
```

```
        BREAK          // end of video / stream lost
```

```
    END IF
```

```
// ---- 2a. PREPROCESSING ----
```

```
resized_frame = RESIZE(frame, width = 800)
```

```
blurred_frame = GAUSSIAN_BLUR(resized_frame, kernel = (5,5))
```

```
gray_frame = CONVERT_TO_GRAYSCALE(blurred_frame)
```

```

// ---- 2b. REGION OF INTEREST MASKING ----
roi_mask = CREATE_MASK(shape = resized_frame.shape, polygon =
ROI_POLYGON)
masked_frame = BITWISE_AND(resized_frame, roi_mask)

// ---- 2c. VEHICLE DETECTION ----
detections = vehicle_detector_model.detect(masked_frame)
// detections = list of (x, y, w, h, class_label, confidence)

FILTER detections WHERE class_label IN {"car","truck","bus","motorbike"}
      AND confidence >= MIN_DETECTION_CONFIDENCE
APPLY NON_MAX_SUPPRESSION(detections, iou_threshold = 0.4)

current_boxes = []
FOR EACH (x, y, w, h, label, confidence) IN detections DO
    centroid = ((x + x + w) / 2, (y + y + h) / 2)
    APPEND {box: (x, y, w, h), centroid: centroid, label: label} TO current_boxes
END FOR

// ---- 2d. MULTI-OBJECT TRACKING (associate detections to existing tracks) ----
unmatched_detections = COPY(current_boxes)
matched_pairs = []

FOR EACH (track_id, track) IN tracker_registry DO
    best_match = NONE
    best_score = 0
    FOR EACH detection IN unmatched_detections DO
        score = COMPUTE_IOU(track.box, detection.box)
        // Alternatively: score based on centroid distance + Kalman prediction
        IF score > best_score AND score > IOU_MATCH_THRESHOLD THEN
            best_score = score
            best_match = detection
        END IF
    END FOR
END FOR

```

```

IF best_match != NONE THEN
    track.box      = best_match.box
    track.centroid = best_match.centroid
    track.disappeared = 0
    track.kalman_filter.UPDATE(best_match.centroid) // smooth trajectory
    APPEND (track_id, best_match) TO matched_pairs
    REMOVE best_match FROM unmatched_detections
ELSE
    track.disappeared = track.disappeared + 1
    track.centroid = track.kalman_filter.PREDICT() // keep predicting briefly
END IF
END FOR

// ---- 2e. REGISTER NEW TRACKS FOR UNMATCHED DETECTIONS ----
FOR EACH detection IN unmatched_detections DO
    new_track = {
        id: next_id,
        box: detection.box,
        centroid: detection.centroid,
        prev_centroid: detection.centroid,
        disappeared: 0,
        kalman_filter: NEW_KALMAN_FILTER(init = detection.centroid)
    }
    tracker_registry[next_id] = new_track
    next_id = next_id + 1
END FOR

// ---- 2f. REMOVE STALE TRACKS ----
FOR EACH (track_id, track) IN tracker_registry DO
    IF track.disappeared > MAX_DISAPPEARED_FRAMES THEN
        REMOVE track_id FROM tracker_registry
    END IF
END FOR

// ---- 2g. LINE-CROSSING COUNTING LOGIC (avoid double counting) ----

```

```

FOR EACH (track_id, track) IN tracker_registry DO
    IF track_id NOT IN counted_ids THEN
        crossed = LINE_CROSSED(track.prev_centroid, track.centroid, COUNT_LINE)
        IF crossed THEN
            vehicle_count = vehicle_count + 1
            ADD track_id TO counted_ids    // ensures each ID counted only once
        END IF
    END IF
    track.prev_centroid = track.centroid
END FOR

```

```

// ---- 2h. VISUALIZATION ----
DRAW_POLYGON(frame, ROI_POLYGON, color = YELLOW)
DRAW_LINE(frame, COUNT_LINE.start, COUNT_LINE.end, color = RED, thickness =
2)

```

```

FOR EACH (track_id, track) IN tracker_registry DO
    (x, y, w, h) = track.box
    DRAW_RECTANGLE(frame, (x, y), (x + w, y + h), color = GREEN, thickness = 2)
    PUT_TEXT(frame, "ID " + track_id, position = (x, y - 8))
    DRAW_CIRCLE(frame, track.centroid, radius = 3, color = BLUE)
END FOR

```

```

PUT_TEXT(frame, "Vehicle Count: " + vehicle_count, position = (20, 40),
    font_scale = 1.0, color = WHITE, background = BLACK)

```

```

// ---- 2i. DISPLAY ----
DISPLAY(frame, window_name = "Vehicle Tracking & Counting")

```

```

// ---- 2j. EFFICIENT LOOP TERMINATION ----
IF KEY_PRESSED('q') THEN
    BREAK
END IF
IF END_OF_VIDEO(video_stream) THEN
    BREAK

```

```
END IF

END WHILE

// ----- 3. CLEANUP -----
video_stream.RELEASE()
DESTROY_ALL_WINDOWS()
PRINT "Final Vehicle Count: " + vehicle_count

END VehicleDetectionTrackingCountingSystem
```

2.3 Key Design Notes

- Restricting detection/tracking to the ROI reduces false positives from irrelevant background motion.
- A centroid/IOU tracker (optionally backed by a Kalman filter, or a full SORT/DeepSORT implementation) maintains consistent IDs across frames even with brief occlusion.
- Double counting is prevented by checking line crossing only once per unique track ID and recording counted IDs in a set.
- Comparing the previous and current centroid position relative to the count line (sign change test) reliably detects a genuine line crossing rather than mere proximity.
- Stale tracks (vehicles that left the frame) are pruned after a disappearance threshold to keep the tracker registry efficient.
- Non-Maximum Suppression removes duplicate/overlapping detections of the same vehicle before tracking begins.